

SmartLost & Found

Cloud-Based Item Recovery System

Abhinaya Lakshmi S - 230701004

Aboorvan SB - 230701011

Awinthika Santhanam - 230701048

Introduction

Lost and misplaced items are a common issue in environments such as colleges, offices, and public places. Traditional lost-and-found systems are manual, inefficient, and lack proper organization.

This project focuses on developing a cloud-based Lost and Found Portal that allows users to report lost and found items in a centralized system. By leveraging cloud computing, the system ensures accessibility, scalability, and real-time data management.

Objective

To build a centralized platform for reporting lost and found items

To deploy the application on cloud using Microsoft Azure

To ensure data availability and accessibility from anywhere

To implement a smart matching system to suggest possible item matches

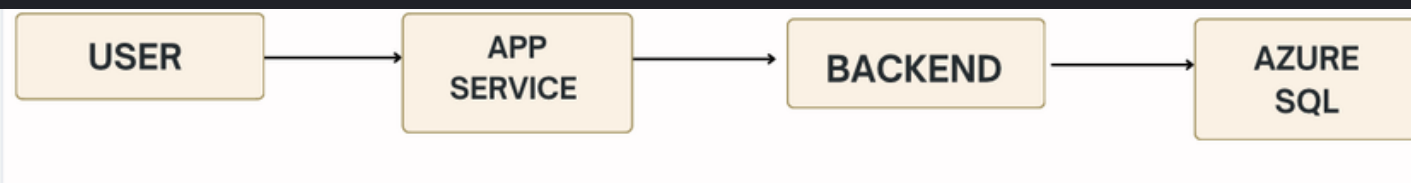
DEPLOYMENT – ARCHITECTURE & MODEL

Architecture Model



3 Tier Architecture

The presentation layer (frontend) handles the user interface using technologies like HTML, CSS, and JavaScript. The application layer (backend) processes user requests, implements business logic, and communicates between the frontend and database using Node.js and Express. The data layer stores and manages data using systems like Azure SQL Database.



System Design

Frontend

HTML, CSS, JavaScript

Allows users to submit and view items

Backend

Node.js with Express

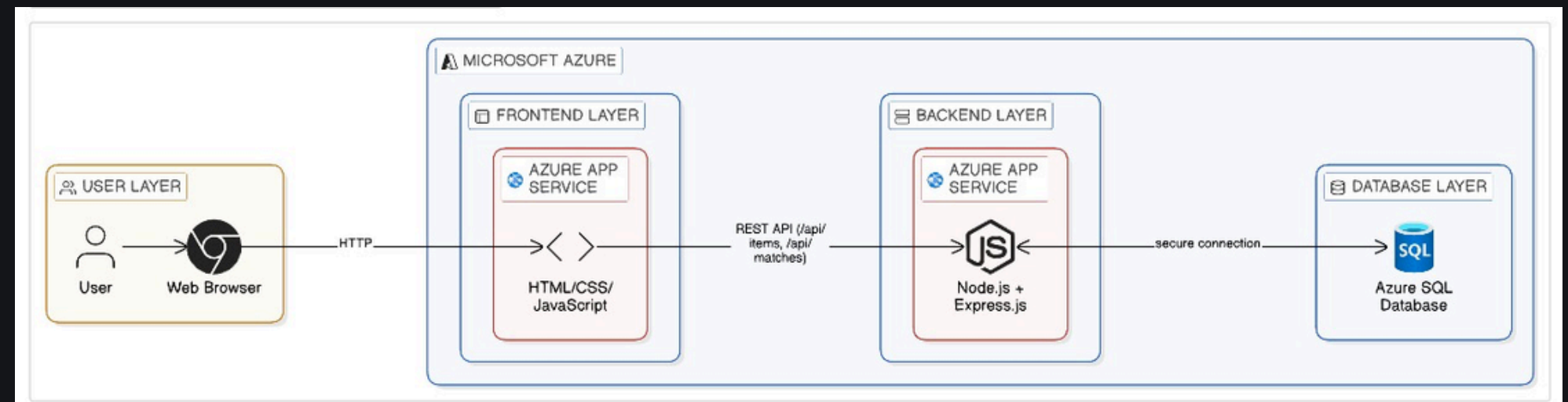
Handles API requests and processing

Database

Azure SQL Database

Stores all item details

Hosted using Azure App Service



DEPLOYMENT – HA, SECURITY & MONITORING

High Availability

Azure App Service ensures high uptime

Application accessible via internet

Reliable cloud infrastructure

Security

Azure SQL protected using firewall rules

Access restricted to specific IP addresses

Secure communication using HTTPS

Monitoring

Azure Monitor for tracking performance

Logging of errors and requests

Easy debugging and maintenance

INFRASTRUCTURE REQUIREMENTS

Infrastructure Requirements refer to the essential resources needed to build and run the application efficiently. These include functional requirements such as adding and viewing items, storing data, and implementing a matching system. It also includes non-functional requirements like scalability, performance, availability, and security. In this project, cloud infrastructure using Microsoft Azure ensures reliable performance, 24/7 accessibility, and secure data handling.

Functional Requirements

- Add lost items
- Add found items
- View all items
- Smart matching system
- Store data in cloud

Non-Functional Requirements

Scalability: Cloud-based deployment

Performance: Fast API response

Availability: 24/7 access

Security: Firewall & HTTPS

Cost: Free-tier Azure usage

AZURE SERVICES MAPPING

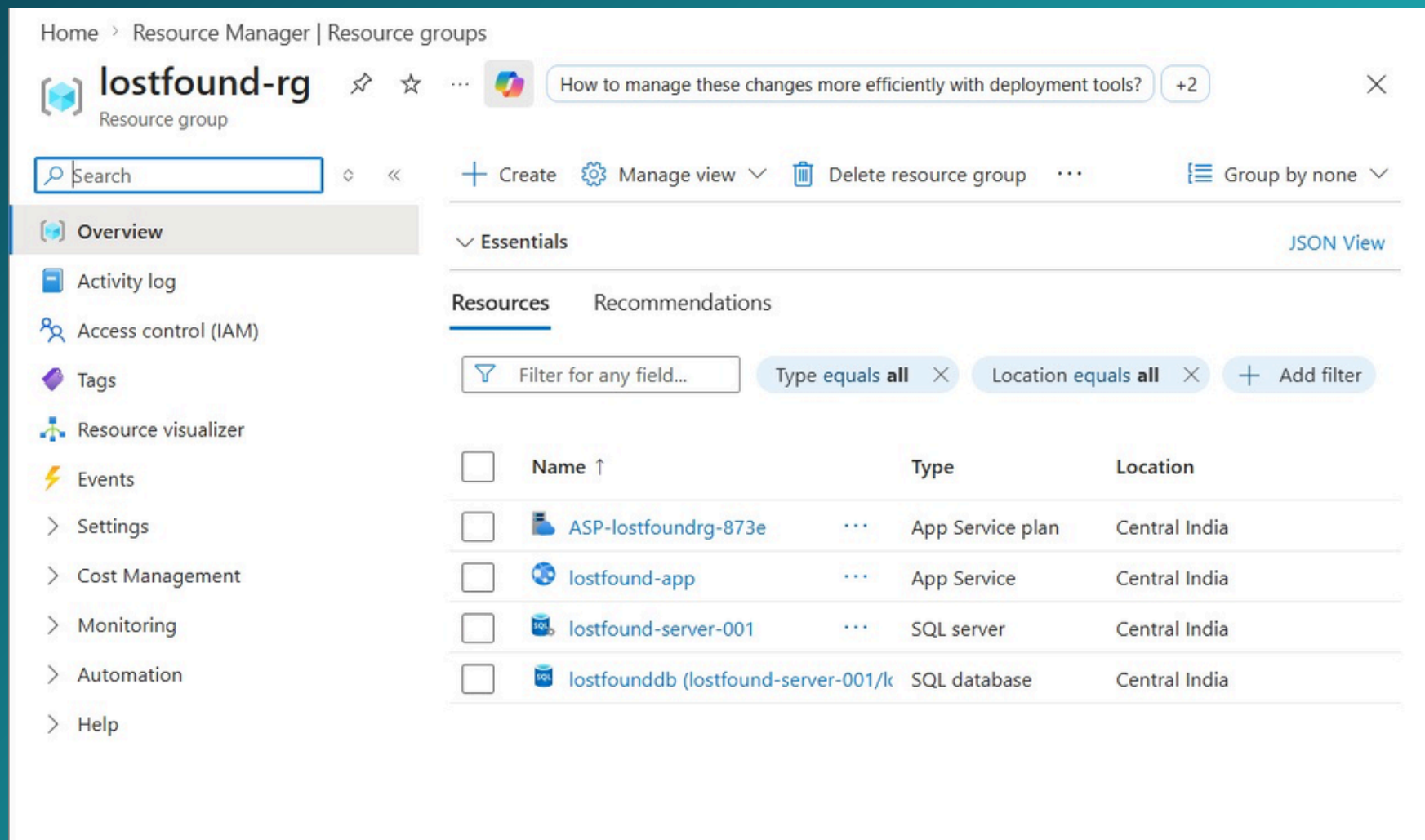


Fig 6.1: Azure Services

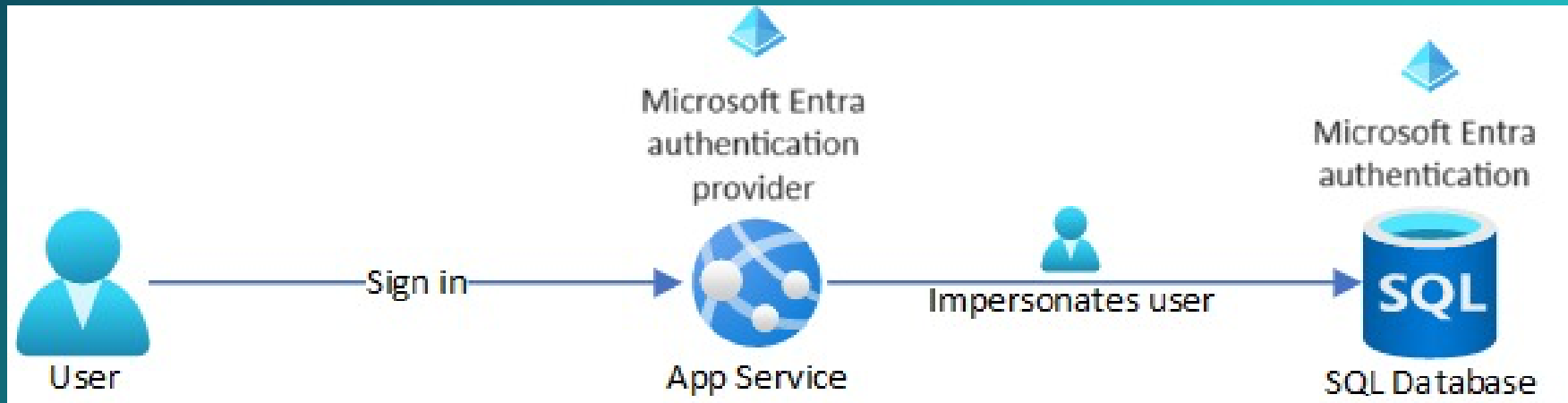


Fig 6.2: Azure Service Flow

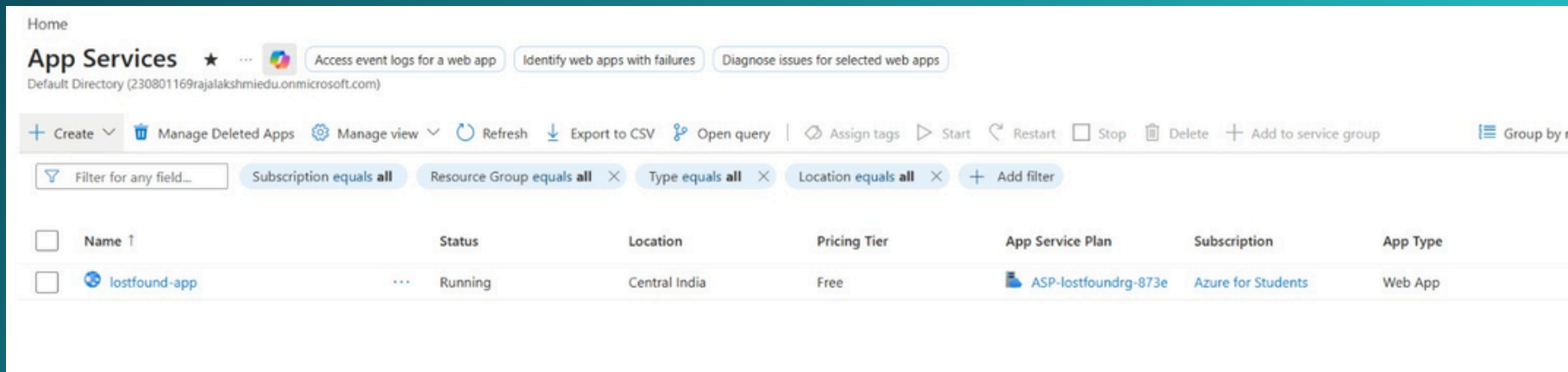


Fig 6.3: Backend + Frontend deployed using Azure App Service

SERVICE JUSTIFICATION

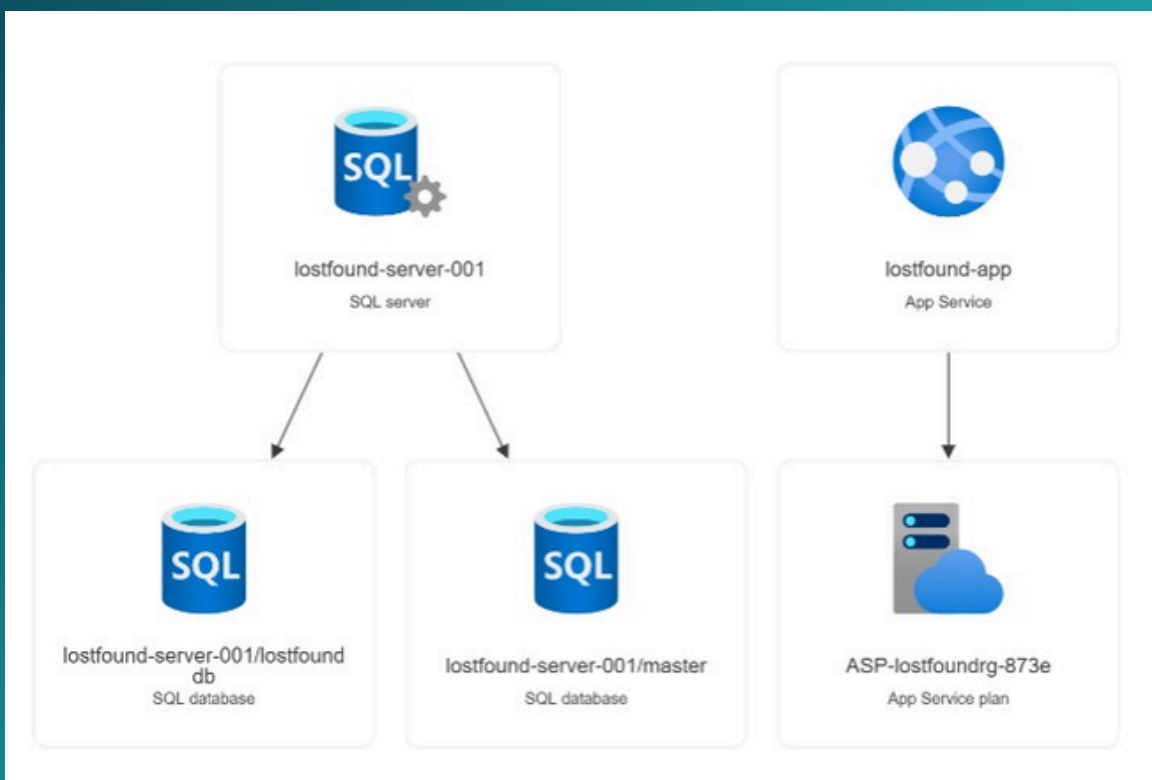


Fig 7.1: Resource Groups

The Resource Group helps in organizing and managing all Azure resources in a structured way, making deployment and monitoring easier.

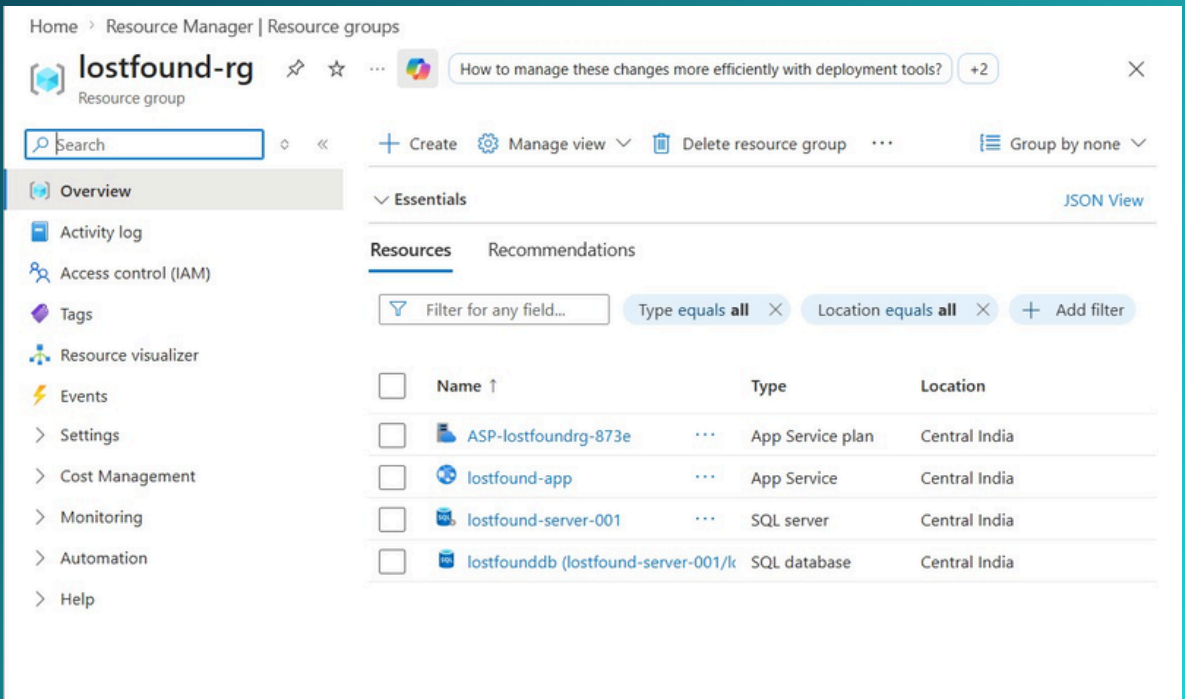


Fig 6.1: Azure Services

Azure App Service is used for hosting because it allows easy deployment, automatic scaling, and eliminates the need for managing servers.

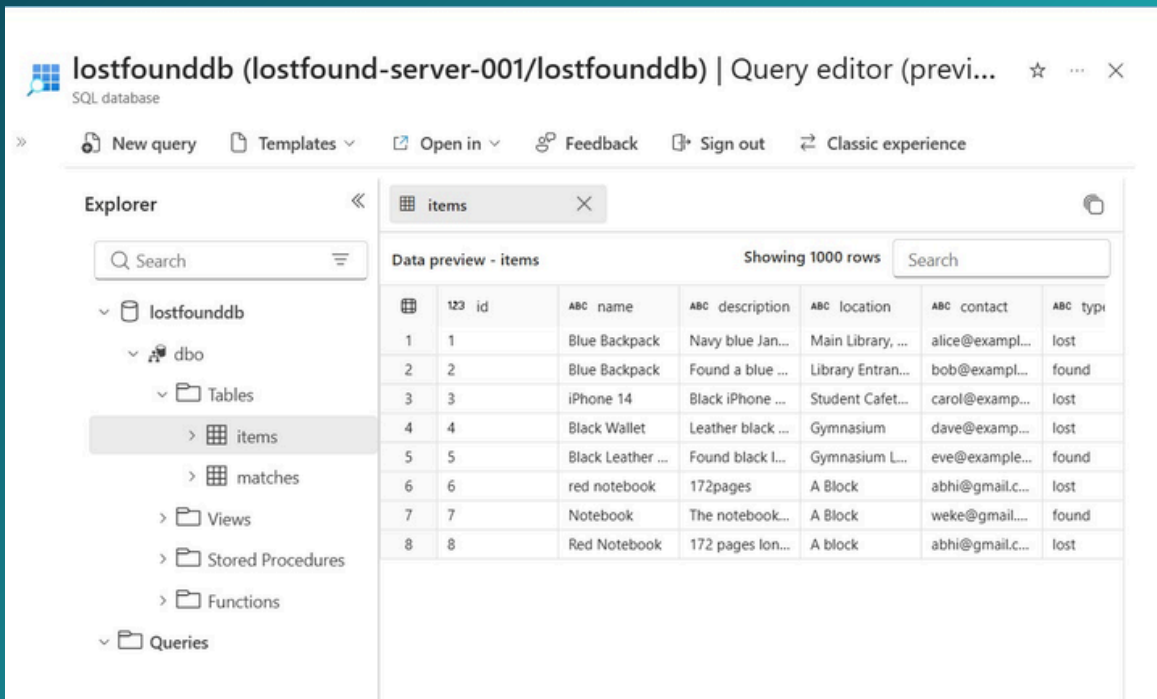
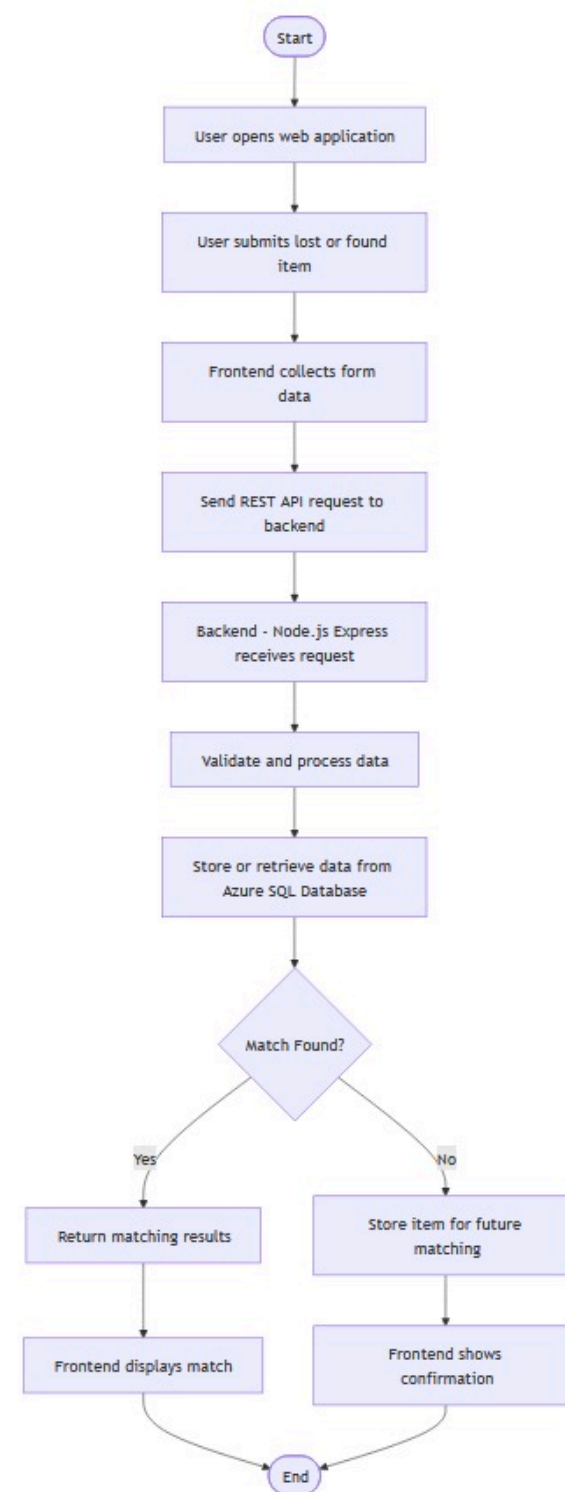


Fig 7.2: Database

Azure SQL Database is chosen for its ability to provide scalable, reliable, and managed data storage with built-in backup and security features.

USER WORK FLOW DIAGRAM



This workflow diagram illustrates how the Lost & Found system processes user interactions from start to end. The process begins when a user accesses the web application and submits details of a lost or found item through a form. The frontend collects this information and sends it to the backend via REST API requests. The backend, built using Node.js and Express, validates and processes the data before interacting with the Azure SQL Database to store or retrieve relevant records. The system then checks for potential matches between lost and found items. If a match is found, the results are returned and displayed to the user; otherwise, the data is stored for future matching and a confirmation message is shown. This workflow ensures efficient data handling, real-time processing, and reliable matching within a cloud-based environment.

SYSTEM FEATURES

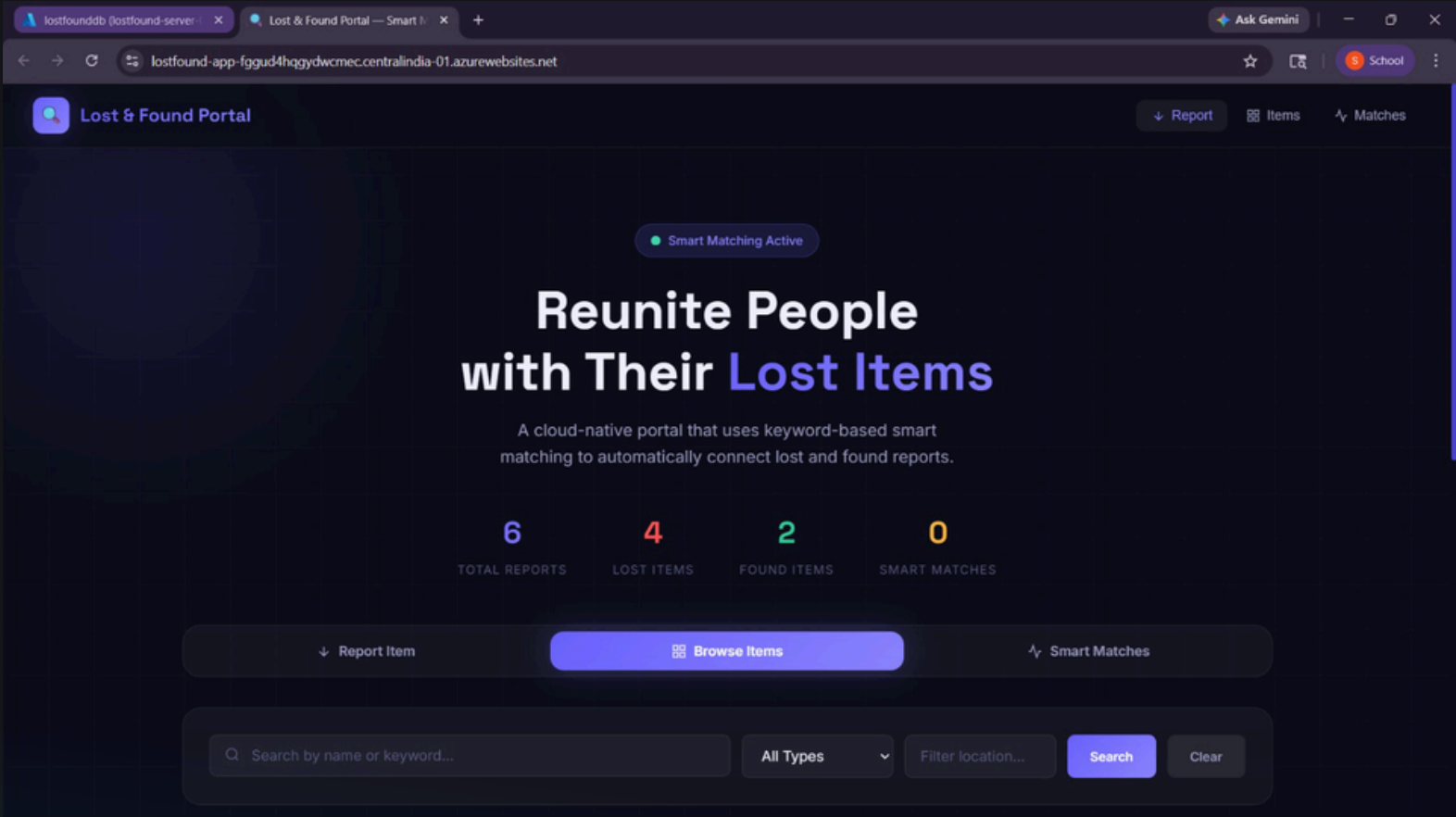


Fig 8.1: Home Screen

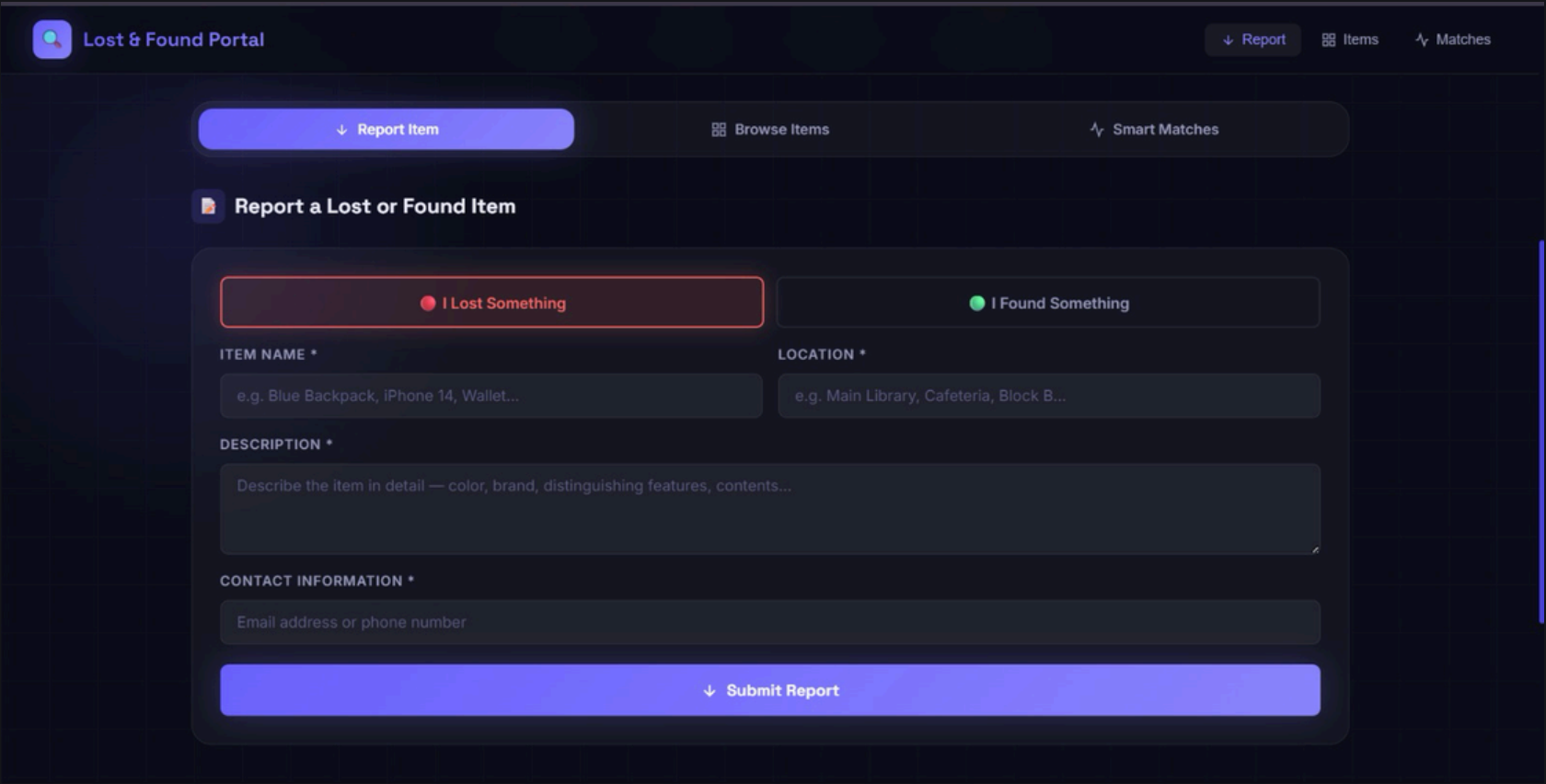


Fig 8.2: Report Lost Item Screen

SYSTEM FEATURES

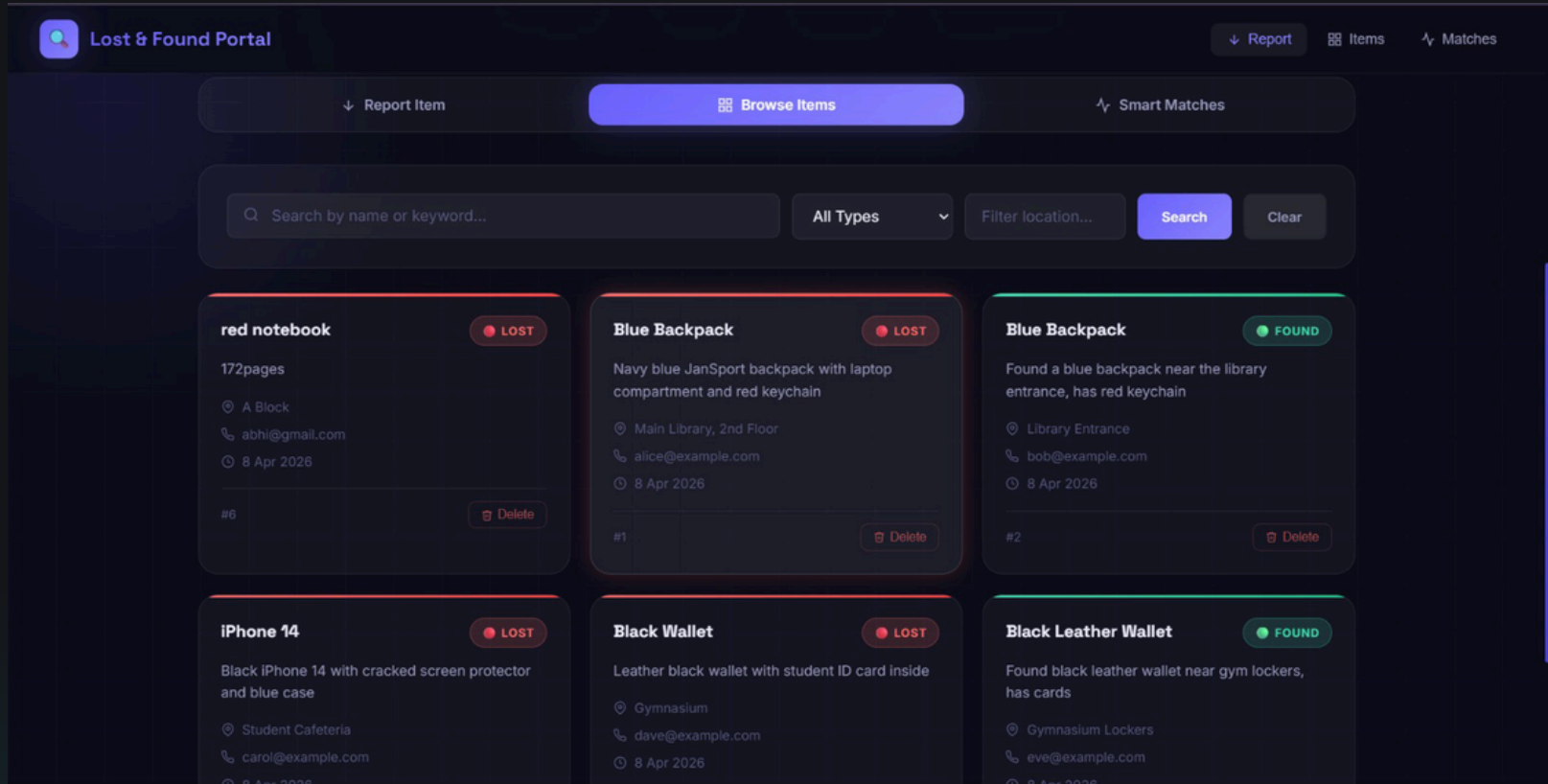


Fig 8.3: Browse Items Screen

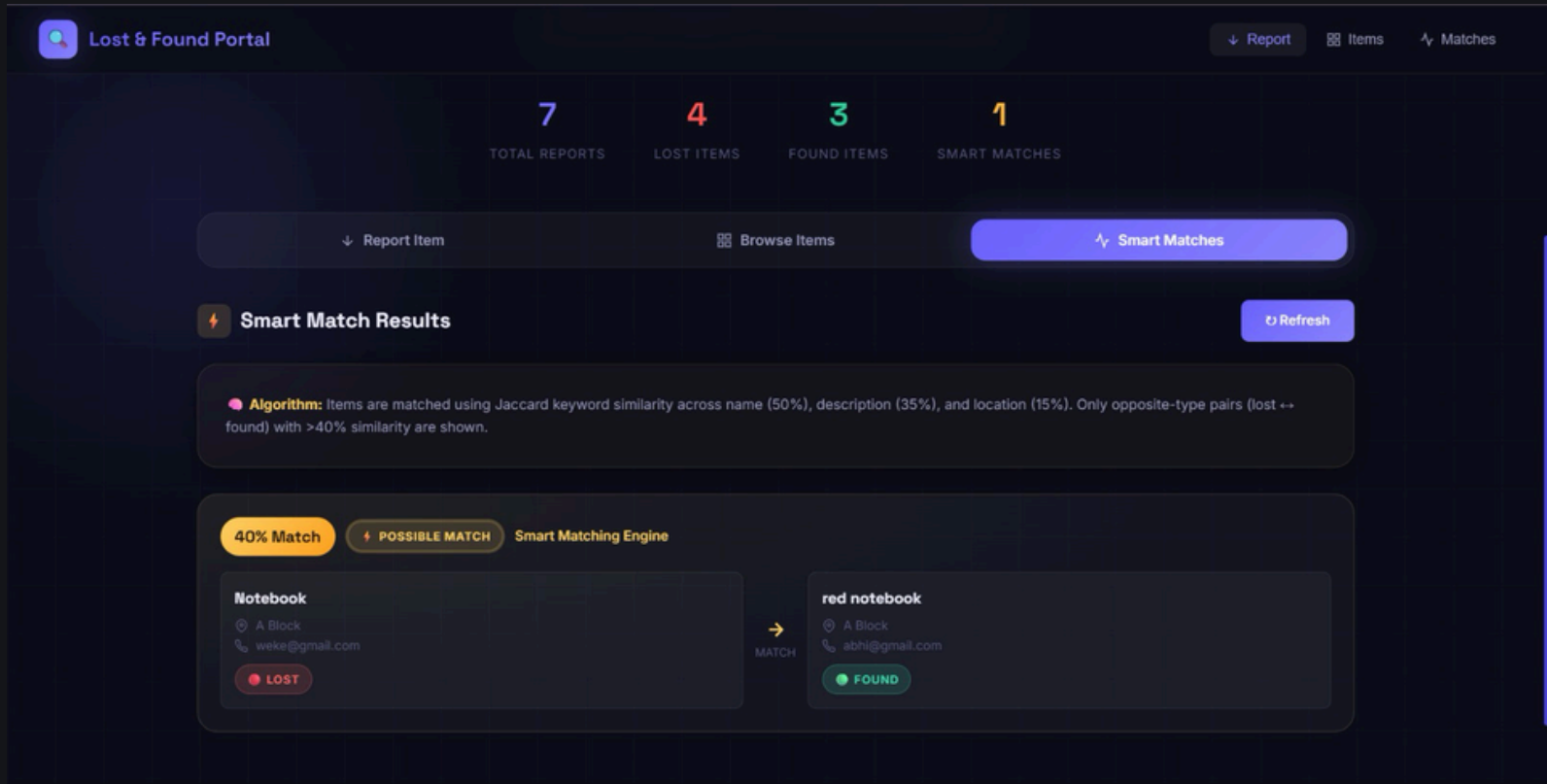


Fig 8.4: Smart Match Screen

thank you